Section 19 : I/O and Streams

Information

- Title: `I/O and Streams`
- Authors: `Etienne KOA`

Aperçu de la section

- Flux et E/S
- Manipulateurs de flux
- Lire et écrire dans un fichier texte
- Utilisation de flux de chaînes

Fichiers, flux et E/S

- `C++` utilise les flux comme interface entre le programme et les périphériques d'entrée et de sortie
- Indépendant de l'appareil lui-même
- Séquence d'octets
- Le flux d'entrée fournit des données au programme
- Le flux de sortie reçoit les données du programme

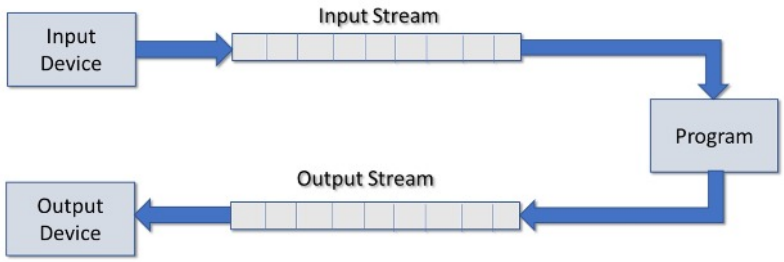

Figure 1: I/O_and_Streams

Fichiers d'en-tête communs

Classes de flux couramment utilisées

Objets de flux global

- Objets globaux - initialisés avant l'exécution principale.
- La meilleure pratique consiste à utiliser `cerr` pour les messages d'erreur et `clog` pour les messages de journal.

Header File	Description
`iostream`	Provides definitions for formatted input and output from/to streams
`fstream`	Provides definitions for formatted input and output from/to file streams
`iomanip`	Provides definitions for manipulators used to format stream I/O

Figure 2: I/O_and_Streams

Class	Description
`ios`	Provides basic support for formatted and unformatted I/O operations. Base class most other classes
`ifstream`	Provides for high-level input operations on file based streams
`ofstream`	Provides for high-level output operations on file based streams
`fstream`	Provides for high-level I/O operations on file based streams Derived from `ofstream` and `ifstream`
`stringstream`	Provides for high-level I/O operations on memory based strings Derived from `istringstream` and `ostringstream`

Figure 3: I/O_and_Streams

Object	Description
`cin`	Standard input stream – by default 'connected' to the standard input device (keyboard) instance of `istream`
`cout`	Standard output stream – by default 'connected' to the standard output device (console) instance of `ostream`
`cerr`	Standard error stream – by default 'connected' to the standard error device (console) Instance of `ostream (unbuffered)`
`clog`	Standard log stream – by default 'connected' to the standard log device (console) Instance of `ostream (unbuffered)`

Figure 4: I/O_and_Streams

Manipulateurs de flux

- Les flux ont des fonctions membres utiles pour contrôler le formatage
- Peut être utilisé sur les flux d'entrée et de sortie
- Le temps de l'effet sur le flux varie
- Peut être utilisé comme fonctions membres ou comme manipulation

```
std::count.width(10);   /: member function
std::cout << std::setw(10); // manipulator
```

- Nous nous concentrerons sur l'utilisation du manipulateur

Common stream manipulators

- **Booléen**
  - `boolalpha, noboolalpha`
- **Entier**
  - `dec, hex, oct, showbase, noshowbase, showpos, noshowpos, uppercase, nouppercase`
- **Point flottant**
  - `fixed, scientific, setprecision, showpoint, noshowpoint, showpos, noswhopos`
- **Largeur du champ, justification et remplissage**
  - `setw, left, right, internal, setfill`
- **Autres**
  - `endl, flush, skipws, noskipws, ws`

Stream Manipulators - boolean

Formatage des types booléens

- La valeur par défaut lors de l'affichage des valeurs booléennes est 1 ou 0.
- Parfois, les chaînes true ou false sont plus appropriées

```
std::cout << (10 == 10) << std::endl;
std::cout << (10 == 20) << std::endl;

// Will display

1
0
```

```
std::cout << std::boolalpha;

std::cout << (10 == 10) << std::endl;
std::cout << (10 == 20) << std::endl;

// Will display

true
false
```

- Toutes les autres sorties booléennes seront affectées

```
std::cout << std::noboolalpha;// 1 or 0
std::cout << std::boolalpha;  // true or false
```

Figure 5: I/O_and_Streams

- Method version

```
std::cout.setf(std::ios::boolalpha);
std::cout.setf(std::ios::noboolalpha);
```

- Reset to default

```
std::cout << std::resetiosflags(std::ios::boolalpha);
```

Figure 6: I/O_and_Streams

Manipulateurs de flux - nombres entiers

Formatage des types entiers

- La valeur par défaut lors de l'affichage de valeurs entières est :
  - `noshowbase` - préfixe utilisé pour afficher le format hexadécimal ou octal
  - `nouppercase` - lors de l'affichage des valeurs `prefix` et `hex`, elles seront en minuscules.
  - `noshowpos` - aucun `+` n'est affiché pour les nombres positifs
- Ces manipulateurs affectent toutes les sorties ultérieures vers le flux

```
int num {255};

std::cout << std::dec << num << std::endl;
std::cout << std::hex << num << std::endl;
std::cout << std::oct << num << std::endl;

// Will display
255
ff
377
```

Figure 7: I/O_and_Streams

Formatage des types entiers - base de définition

Formatage des types entiers - affichage de la base

```
int num {255};

std::cout << std::showbase;  // std::noshowbase
std::cout << std::dec << num << std::endl;
std::cout << std::hex << num << std::endl;
std::cout << std::oct << num << std::endl;

// Will display
255
0xff           // note the 0x prefix for hexadecimal
0377           // note the 0 prefix for octal
```

Figure 8: I/O_and_Streams

Formatage des types entiers - affichage hexadécimal en majuscule

Formatage des types entiers - affichage du signe positif

Définition/réinitialisation des types entiers

Manipulateurs de flux - Virgule flottante

Formatage des types à virgule flottante

- La valeur par défaut lors de l'affichage des valeurs à virgule flottante est :
  - `setprecision` - nombre de chiffres affichés (6).
  - `fixed` - non fixe à un nombre ou à des chiffres spécifiques après la virgule décimale
  - `noshowpoint` - les zéros à droite ne sont pas affichés

```
int num {255};

std::cout << std::showbase << std::uppercase;
std::cout << std::hex << num << std::endl;

// Will display

0XFF            // note capitalized XFF
```

Figure 9: I/O_and_Streams

```
int num1 {255};
int num2 {-255};

std::cout << num1 << std::endl;   // 255
std::cout << num2 << std::endl;   // -255

std::cout << std::showpos;        // std::noshowpos

std::cout << num1 << std::endl;   // +255
std::cout << num2 << std::endl;   // -255
```

Figure 10: I/O_and_Streams

- Set using setf

```
std::cout.setf(std::ios::showbase);
std::cout.setf(std::ios::uppercase);
std::cout.setf(std::ios::showpos);
```

- Reset to defaults

```
std::cout << std::resetiosflags(std::ios::basefield);
std::cout << std::resetiosflags(std::ios::showbase);
std::cout << std::resetiosflags(std::ios::showpos);
std::cout << std::resetiosflags(std::ios::uppercase);
```

Figure 11: I/O_and_Streams

  - `nouppercase` - lors de l'affichage en notation scientifique
  - `noshowpos` - aucun `+` n'est affiché pour les nombres positifs

- Ces manipulateurs affectent toutes les sorties ultérieures du flux.

Formatage des types à virgule flottante - précision

```
double num {1234.5678};

std::cout << num << std::endl;

// Will display

1234.57  // Notice precision is 6 and rounding

double num {123456789.987654321};

std::cout << num << std::endl;

// Will display

1.23457e+008     // Notice precision is 6
```

Formatage des types à virgule flottante - corrigé

```
double num {123456789.987654321};

std::cout << std::setprecision(9);
std::cout << num << std::endl;

// Will display

123456790    // Note that rounding occurs
```

Figure 12: I/O_and_Streams

Formatage des types à virgule flottante - fixé

```
 double num {123456789.987654321};

 std::cout << std::fixed;
 std::cout << num << std::endl;

// Will display precision 6 from the decimal

123456789.987654
```

```
 double num {123456789.987654321};

 std::cout << std::setprecision(3) << std::fixed;
 std::cout << num << std::endl;

// Will display precision 3 from the decimal

123456789.988
```

Formatage des types à virgule flottante – scientifique

```
 double num {123456789.987654321};

 std::cout << std::setprecision(3)
           << std::scientific;
 std::cout << num << std::endl;

// Will display precision 3

1.23e+008
```

Figure 13: I/O_and_Streams

Formatage des types à virgule flottante - majuscules scientifiques

Formatage des types à virgule flottante - affichage du signe positif

Formatage des types à virgule flottante - zéros à droite

Retour aux paramètres généraux

Manipulateurs de flux - aligner et remplir

Largeur du champ, aligner et remplir

```
 double num {123456789.987654321};

 std::cout << std::setprecision(3)
           << std::scientific
           << std::uppercase;
 std::cout << num << std::endl;

// Will display precision 3

1.23E+008     // Note the capital 'E'
```

Figure 14: I/O_and_Streams

```
 double num {123456789.987654321};

 std::cout << std::setprecision(3)
           << fixed
           << std::showpos;

 std::cout << num << std::endl;

// Will display

+123456789.988      // Note the leading '+'
```

Figure 15: I/O_and_Streams

```
 double num {12.34};

 std::cout << num << std::endl;       // 12.34

 std::cout << std::showpoint;

 std::cout << num << std::endl;       // 12.3400

// Will display

12.34      // Note no trailing zeroes (default)
12.3400    // Note trailing zeroes up to precision
```

Figure 16: I/O_and_Streams

- unsetf

```
std::cout.unsetf(std::ios::scientific | std::ios:fixed);
```

or

```
std::cout << std::resetiosflags(std::ios::floatfield);
```

- Refer to the docs for other set/reset flags

Figure 17: I/O_and_Streams

- La valeur par défaut lors de l'affichage des valeurs à virgule flottante est :
  - `setw` - non défini par défaut
  - `left` - lorsqu'il n'y a pas de largeur de champ, à droite - lors de l'utilisation de la largeur de champ
  - `fill` - non défini par défaut - un espace vide est utilisé.
- Certains de ces manipulateurs n'affectent que l'élément de données suivant mis sur le flux.

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << num << hello << std::endl;

// Will display

1234.57Hello
```

Défaut

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << std::setw(10) << num
          << hello << std::endl;

// Will display
1234567890123456789012345678901234567890
  1234.57Hello
```

Largeur du champ - `setw`

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << std::setw(10) << num
          << std::setw(10) << hello
          << std::setw(10) << hello << std::endl;

// Will display
1234567890123456789012345678901234567890
   1234.57     Hello     Hello
```

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << std::setw(10)
          << std::left
          << num        // only affects num!
          << hello << std::endl;

// Will display
1234567890123456789012345678901234567890
1234.57   Hello
```

Largeur du champ - `justification`

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << std::setw(10) << num
          << std::setw(10) << std::right << hello
          << std::setw(15) << std::right << hello
          << std::endl;

// Will display
1234567890123456789012345678901234567890
   1234.57     Hello          Hello
```

Largeur du champ - `setw`

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << std::setfill('-');
std::cout << std::setw(10) << num
          << hello << std::endl;

// Will display
1234567890123456789012345678901234567890
---1234.57Hello
```

Largeur du champ de remplissage - `setfill`

```
double num {1234.5678};
std::string hello{"Hello"};

std::cout << std::setfill('*');
std::cout << std::setw(10) << num
          << std::setfill('-') << std::setw(10) << hello
          << std::setw(15) << hello
          << std::endl;

// Will display
1234567890123456789012345678901234567890
***1234.57-----Hello----------Hello
```

Largeur du champ - `setw`

Lecture à partir d'un fichier texte

Fichiers d'entrée (`fstream` et `ifstream`)

1. `#include <fstream>`
2. Déclarez un objet `fstream` ou `ifstream`.
3. Connectez-le à un fichier sur votre système de fichiers (l'ouvre en lecture)
4. Lire les données du fichier via le flux
5. Fermez le flux

Ouverture d'un fichier en lecture avec (`fstream`)

Ouverture d'un fichier en lecture avec (`ifstream`)

Ouvrir un fichier en lecture avec `open`

Vérifiez si le fichier s'est ouvert avec succès (`is_open`)

Vérifiez si le fichier s'est ouvert avec succès - testez l'objet flux

Fermer un fichier + Fermez toujours tous les fichiers ouverts pour éliminer toutes les données non écrites

```
std::fstream in_file {"../myfile.txt",
                      std::ios::in };
```

• Open for reading in binary mode

```
std::fstream in_file {"../myfile.txt",
               std::ios::in | std::ios::binary};
```

Figure 18: I/O_and_Streams

```
std::ifstream in_file {"../myfile.txt",
                      std::ios::in };

std::ifstream in_file {"../myfile.txt"};
```

• Open for reading in binary mode

```
std::ifstream in_file {"../myfile.txt",
                       std::ios::binary};
```

Figure 19: I/O_and_Streams

```
std::ifstream in_file;
std::string filename;
std::cin >> filename;  // get the file name

in_file.open(filename);
// or
in_file.open(filename, std::ios::binary);
```

Figure 20: I/O_and_Streams

```
if (in_file.is_open()) {
   // read from it
} else {
   // file could not be opened
   // does it exist?
   // should the program terminate?
}
```

Figure 21: I/O_and_Streams

```
if (in_file) {    // just check the object
   // read from it
} else {
   // file could not be opened
   // does it exist?
   // should the program terminate?
}
```

Figure 22: I/O_and_Streams

```
in_file.close();
```

Figure 23: I/O_and_Streams

Lecture à partir de fichiers en utilisant `(>>)` + Nous pouvons utiliser l'opérateur d'extraction pour une lecture formatée + De la même manière que nous l'avons utilisé avec `cin`

```
int num {};
double total {};
std::string name {};

in_file >> num;
in_file >> total >> name;
```

```
100
255.67
Larry
```

Figure 24: I/O_and_Streams

Lecture à partir de fichiers à l'aide de `getline` + Nous pouvons utiliser `getline` pour lire le fichier une ligne à la fois

```
std::string line{};

std::getline(in_file, line);
```

```
This is a line
```

Figure 25: I/O_and_Streams

Lecture du fichier texte une ligne à la fois

```
std::ifstream in_file{"../myfile.txt"};     // open file
std::string line {};

if (!in_file) {       // check if file is open
   std::cerr << "File open error" << std::endl;
   return 1;     // exit the program (main)
}
while (!in_file.eof()) {          // while not at the end
   std::getline(in_file, line);   // read a line
   cout << line << std::endl;     // display the line
}
in_file.close();// close the file
```

```
std::ifstream in_file{"../myfile.txt"}; // open file
std::string line {};

if (!in_file) {     // check if file is open
   std::cerr << "File open error" << std::endl;
   return 1;   // exit the program (main)
}
while (std::getline(in_file, line)) // read a line
   cout << line << std::endl;       // display the line

in_file.close();    // close the file
```

Lecture du fichier texte un caractère à la fois (`get`)

```
std::ifstream in_file{"../myfile.txt"}; // open file
char c;

if (!in_file) {     // check if file is open
   std::cerr << "File open error" << std::endl;
   return 1;   // exit the program (main)
}
while (in_file.get(c)) // read a character
   cout << c;                // display the character

in_file.close();    // close the file
```

Figure 26: I/O_and_Streams

Écrire dans un fichier texte

Fichiers de sortie (`fstream` et `ofstream`)

- `fstream` et `ofstream` sont couramment utilisés pour les fichiers de sortie

1. `#include <fstream>`
2. Déclarez un objet `fstream` ou `ofstream`
3. Connectez-le à un fichier sur votre système de fichiers (l'ouvre en écriture)
4. Écrivez des données à un moment donné via le flux
5. Fermez le flux

- Les fichiers de sortie seront créés s'ils n'existent pas
- Les fichiers de sortie seront écrasés (tronqués) par défaut
- Peut être ouvert pour que de nouvelles écritures soient ajoutées
- Peut être ouvert en mode texte ou binaire

```
std::fstream out_file {"../myfile.txt",
                       std::ios::out };
```

• Open for writing in binary mode

```
std::fstream out_file {"../myfile.txt",
                std::ios::out| std::ios::binary};
```

Figure 27: I/O_and_Streams

Ouverture d'un fichier en écriture avec (`fstream`)

Ouverture d'un fichier en écriture avec (`ofstream`)

```
std::ofstream out_file {"../myfile.txt",
                        std::ios::out};

std::ofstream out_file {"../myfile.txt"};
```

• Open for writing in binary mode

```
std::ofstream out_file {"../myfile.txt",
                         std::ios::binary};

// truncate (discard contents) when opening
std::ofstream out_file {"../myfile.txt",
                        std::ios::trunc};

// append on each write
std::ofstream out_file {"../myfile.txt",
                        std::ios::app};

// seek to end of stream when opening
std::ofstream out_file {"../myfile.txt",
                        std::ios::ate};
```

Ouverture d'un fichier en écriture avec open

```
std::ofstream out_file;
std::string filename;
std::cin >> filename;  // get the file name

out_file.open(filename);
// or
out_file.open(filename, std::ios::binary);
```

Vérifiez si le fichier s'est ouvert avec succès (`is_open`)

```
if (out_file.is_open()) {
   // read from it
} else {
   // file could not be created or
   // does it exist?
   // should the program terminate?
}
```

Vérifiez si le fichier s'est ouvert avec succès - testez l'objet flux

```
if (out_file) {  // just check the object
   // read from it
} else {
   // file could not be opened
   // does it exist?
   // should the program terminate?
}
```

Fermer un fichier + Fermez toujours tous les fichiers ouverts pour éliminer toutes les données non écrites

```
out_file.close();
```

Figure 28: I/O_and_Streams

Lecture à partir de fichiers en utilisant (`<<`) + Nous pouvons utiliser l'opérateur d'insertion pour l'écriture formatée + De la même manière que nous l'avons utilisé avec `cout`

Copier un fichier texte une ligne à la fois

```
int num {100};
double total {255.67};
std::string name {"Larry"};

out_file << num   << "\n"
         << total << "\n"
         << name  << std::endl;
```

```
100
255.67
Larry
```

Figure 29: I/O_and_Streams

```
std::ifstream in_file{"../myfile.txt"}; // open file
std::ofstream out_file("../copy.txt"};

if (!in_file) {     // check if file is open
   std::cerr << "File open error" << std::endl;
   return 1;   // exit the program (main)
}
if (!out_file) {         // check if file is open
   std::cerr << "File create error" << std::endl;
   return 1;   // exit the program (main)
}

std::string line {};

while (std::getline(in_file, line)) // read a line
   out_file << line << std::endl;   // write a line

in_file.close();    // close the files
out_file.close();
```

Copier un fichier texte une ligne à la fois (get/put)

```
std::ifstream in_file{"../myfile.txt"}; // open file
std::ofstream out_file("../copy.txt"};

if (!in_file) {     // check if file is open
   std::cerr << "File open error" << std::endl;
   return 1;   // exit the program (main)
}
if (!out_file) {         // check if file is open
   std::cerr << "File create error" << std::endl;
   return 1;   // exit the program (main)
}
```

```
char c;

while (in_file.get(c)) // read a character
   out_file.put(c);    // write the character

in_file.close();    // close the files
out_file.close();
```

Utilisation de flux de chaînes

`stringstream, istringstream et ostringstream`

1. `#include <sstream>`
2. Déclarez un objet `stringstream, istringstream ou ostringstream`
3. Connectez-le à un `std::string`
4. Lire/écrire des données depuis/vers le flux de chaînes à l'aide d'E/S formatées

Lecture à partir d'un `stringstream`

```
#include <sstream>

int num {};
double total {};
std::string name {};
std::string info {"Moe  100  1234.5"};

std::istringstream iss{info};
iss >> name >> num >> total;
```

Figure 30: I/O_and_Streams

Écrire dans un `stringstream`

Validation de l'entrée avec `stringstream`

Emplacements de fichiers avec certains IDE populaires

- Emplacement des fichiers et IDE

```
#include <sstream>

int num {100};
double total {1234.5};
std::string name {"Moe"};

std::ostringstream oss{};
oss << name << " " << num << " " << total;
std::cout << oss.str() << std::endl;
```

Figure 31: I/O_and_Streams

```
int value {};
std::string input{};

std::cout << "Enter an integer: ";
std::cin >> input;

std::stringstream ss{input};
if (ss >> value) {
   std::cout << "An integer was entered";
else
   std::cout << "An integer was NOT entered";
```

Figure 32: I/O_and_Streams

IDE	File Path
CodeLite	`std::ifstream in_file {"../test.txt"};`
Windows Visual Studio	`std::ifstream in_file {"test.txt"};`
Code::Blocks	`std::ifstream in_file {"test.txt"};`
CLion	`std::ifstream in_file {"../test.txt"};`
Xcode	See the video

Figure 33: I/O_and_Streams